Tutorial: Landau Notation (2) + ADT

Institute of
Algorithms and Theory

Remark: Goal of the Previous Tutorial Session

Landau Notation (1)

- Fluent with the definitions
- Proving statements like:
  - $\log^2 n = O(n)$
- Get a feeling for asymptotic growth
- Sorting functions: What grows faster?

Remark

Big-O (O): $O(g) = \{ \mathrm{f} \mid$ there exist constants $c > 0$ and n_0 such that for all $n \geq n_0$,
$$|f(n)| \leq c \cdot |g(n)| \}$$
Interpretation: g is an upper bound on growth rate.

If we remove the equality.....

Big-Omega (Ω): $\Omega(g) = \{ \mathrm{f} \mid$ there exist constants $c > 0$ and n_0 such that for all $n \geq n_0$,
$$|f(n)| \geq c \cdot |g(n)| \}$$
Interpretation: g is a lower bound on growth rate.

Big-Theta (Θ): $\Theta(g) = \{ \mathrm{f} \mid$ there exist constants $c_1, c_2 > 0$ and n_0 such that for all $n \geq n_0$,
$$c_1 \cdot |g(n)| \leq |f(n)| \leq c_2 \cdot |g(n)| \}$$
Interpretation: g has asymptotically the same growth rate.

Formally, $O(g), \Omega(g), \Theta(g)$ are **sets** of functions, but one usually misuses notation and writes
$$f = O(g),\ f = \Omega(g), \text{ and } f = \Theta(g)$$

One-sided "equality". $f = O(g)$ does not automatically mean $g = O(f)$

Little-o and little-ω

Little-o (o): $o(g(n)) = \{\text{function f} \mid$ for every $\varepsilon > 0$, there exists an n_0 such that for all $n \geq n_0$, $|f(n)| < \varepsilon \cdot |g(n)|\}$

Interpretation: Strict upper bound (grows strictly slower).

Little-omega (ω): $\omega(g(n)) = \{\text{f} \mid$ for every $\varepsilon > 0$, there exists an n_0 such that for all $n \geq n_0$, $|f(n)| > \varepsilon \cdot |g(n)|\}$

Interpretation: Strict lower bound (grows strictly faster).

What is true?

- $f = O(g) \Rightarrow f = o(g)$?
- $f = \omega(g) \Rightarrow f = \Omega(g)$?

What about the other directions?

Landau-Notation: Equivalent Characterization

Big-O(O): $f(n) = O(g(n))$ iff

$$\limsup_{n\to\infty} \frac{|f(n)|}{|g(n)|} < \infty$$

Big-Omega (Ω): $f(n) = \Omega(g(n))$ iff

$$\liminf_{n\to\infty} \frac{|f(n)|}{|g(n)|} > 0$$

Theta (Θ): $f(n) = \Theta(g(n))$ iff

$$0 < \liminf_{n\to\infty} \frac{|f(n)|}{|g(n)|} \leq \limsup_{n\to\infty} \frac{|f(n)|}{|g(n)|} < \infty$$

Little-o (o): $f(n) = o(g(n))$ iff

$$\lim_{n\to\infty} \frac{|f(n)|}{|g(n)|} = 0$$

Little-omega (ω): $f(n) = \omega(g(n))$ iff

$$\lim_{n\to\infty} \frac{|f(n)|}{|g(n)|} = \infty$$

What does $o(1)$ and $\omega(1)$ mean?

$f(n) = o(1)$ iff

$$\lim_{n\to\infty} \frac{|f(n)|}{|1|} = 0$$

$f(n) = \omega(1)$ iff

$$\lim_{n\to\infty} \frac{|f(n)|}{|1|} = \infty$$

Example: Landau Notation

Show that $\log n = \mathrm{o}(n)$ for both different definitions of the o-notation.

What does that mean?

Def 1) For **all** $\varepsilon > 0$, there exists n_0 such that $\log \ n < \ c \cdot n$ for any $n > n_0$.

Def 2) $\lim_{n \to \infty} \frac{|\log \ n|}{|n|} = 0$

Example: Landau Notation

Show that $\log n = \mathrm{o}(n)$ for both different definitions of the o-notation.

Def 1) For **all** $\varepsilon > 0$, there exists n_0 such that $\log \quad n < \varepsilon \cdot n$ for any $n > n_0$.

We know that $\log n < n^{1/2}$ for any $n \geq 10$.

For a fixed $\varepsilon > 0$, let $n_0 = \max\{1/\varepsilon^2, 10\}$.

Then we have

$$\log n < n^{1/2} = n^{-1/2} \cdot n < \varepsilon \cdot n$$

Example: Landau Notation

Show that $\log n = \mathrm{o}(n)$ for both different definitions of the o-notation.

Def 2) $\lim\limits_{n\to\infty} \frac{|\log n|}{|n|} = 0$

Use L'Hôpital Rule,

$$\lim_{n\to\infty} \frac{|\log n|}{|n|} = \lim_{n\to\infty} \frac{\left|\frac{1}{n}\right|}{|1|} = \lim_{n\to\infty} \frac{1}{|n|} = 0$$

$$= \text{"}\frac{\infty}{\infty}\text{"}$$

Logarithmic vs Polynomial Growth

What happens for „general“ polynomials?

Is there a constant $p > 0$ such that $\log^{\mathrm{p}} \mathrm{n} = \Omega(n)$? No

Claim: $\log^{\mathrm{p}} \mathrm{n} = o(n)$ for all positive integer $p > 0$.

Proof: Let $p > 0$ be an arbitrary fixed positive integer.

$$\lim_{n\to\infty} \frac{|\log^{\mathrm{p}} n|}{|n|} = \lim_{n\to\infty} \frac{|\frac{\mathrm{p}}{\mathrm{n}}\log^{\mathrm{p}-1} n|}{1} = \lim_{n\to\infty} \frac{|p\log^{\mathrm{p}-1} n|}{|n|} = \ldots = \lim_{n\to\infty} \frac{p!}{n} = 0$$

(Use L’Hôpital Rule recursively)

Polynomials grow faster!

Abstract Data Structure/Abstract Data Type (ADT)

Abstract Data Structure/Abstract Data Type (ADT):

- Specification which kind of data should be stored
- Specification which operations should be executeable (e.g. insert(int x))
  - With the semantics of each operation

Data Structure:

- Explicit implementation of an ADT
- Cost/Runtime of each operation might vary depending on the implementation and or the computational model (e.g. hash tables implemented on a distr. system perfom very differently)

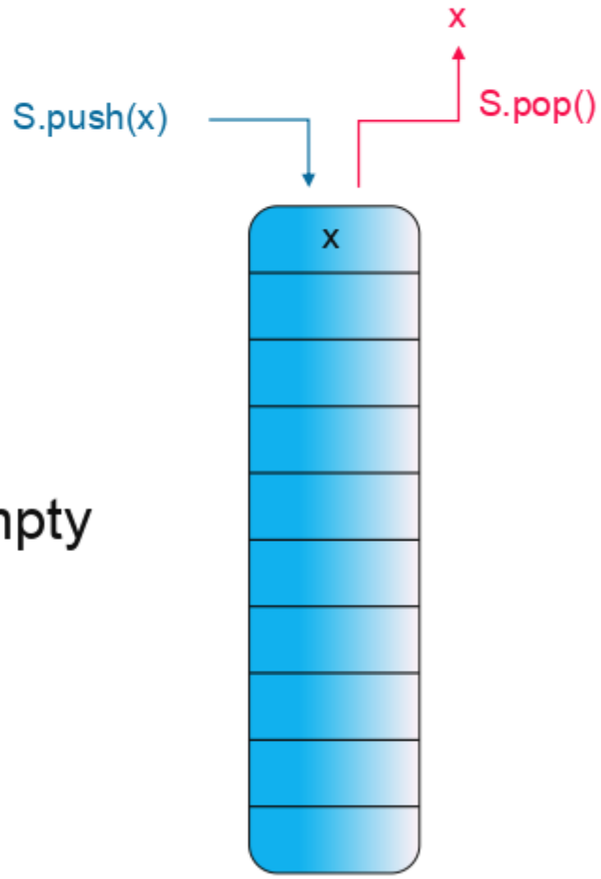
Example: Stack (ADT)

Stack

- Stores a set/sequence of elements

Operations:

- Create: creates an empty Stack
- S.push(x): appends the element x to the stack
- S.pop(): returns the „top most“ element and deletes it
- S.isEmpty: returns bool, depending wheither the stack is empty



LIFO (= last in first out)

only access to the “top” element

Example: Stack (Data Structure)

Array based implementation of Stack:

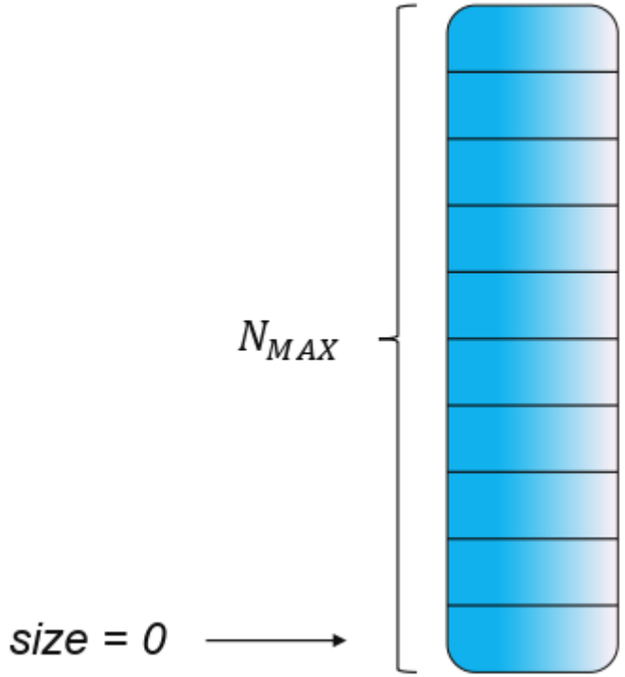
Operations: create, push, pop, isEmpty

Assumption: Stack will not get longer than N_{MAX}

Variables to keep track of the stack:

- *stack*: array of length N_{MAX}
- *size*: number of elements in the stack

```
create():
    stack = new array of length N_MAX
    size = 0

S.isEmpty():
  return (size == 0);
```



Example: Stack (Data Structure)

Array based implementation of Stack:

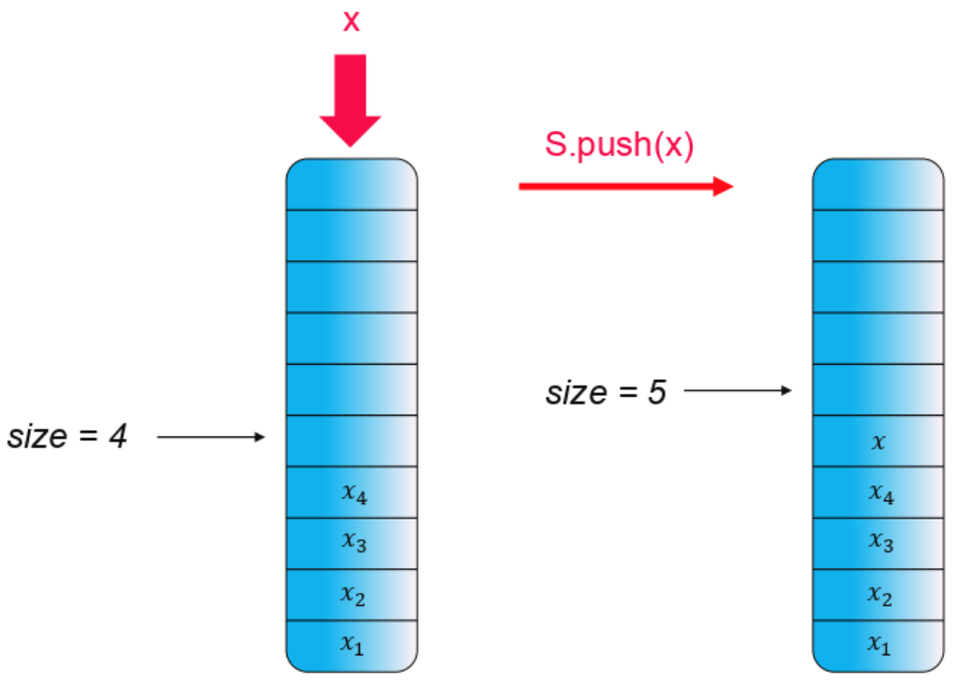
Operations: create, push, pop, isEmpty

Assumption: Stack will not get longer than N_{MAX}

Variables to keep track of the stack:

- *stack*: array of length N_{MAX}
- *size*: number of elements in the stack

```
S.push(x):
  if(size < N_MAX):
      stack[size] = x;
      size += 1;
```



Example: Stack (Data Structure)

Array based implementation of Stack:

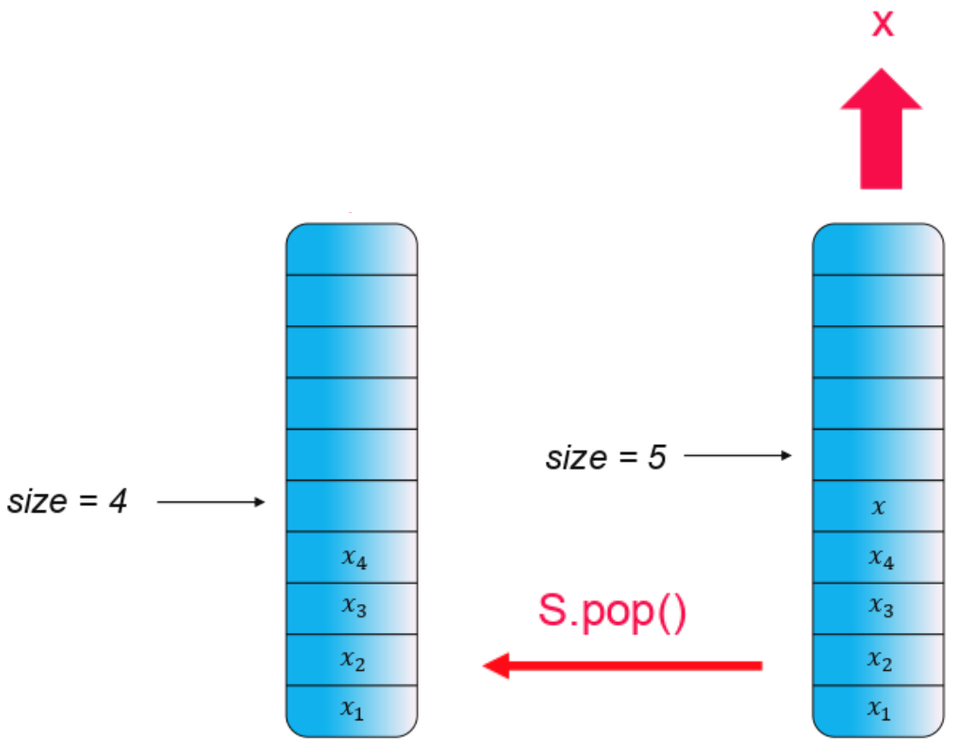
Operations: create, push, pop, isEmpty

Assumption: Stack will not get longer than N_{MAX}

Variables to keep track of the stack:

- *stack*: array of length N_{MAX}
- *size*: number of elements in the stack

```
S.pop():
  if(size == 0):
        return default value; (or return error)
  else:
        size = size - 1;
        return stack[size];
```



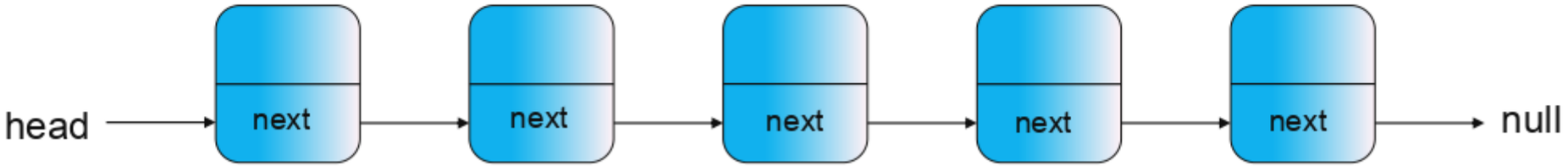
Lists

Want to be able to have **dynamic size**, so we need a delete operation.

List (ADT):

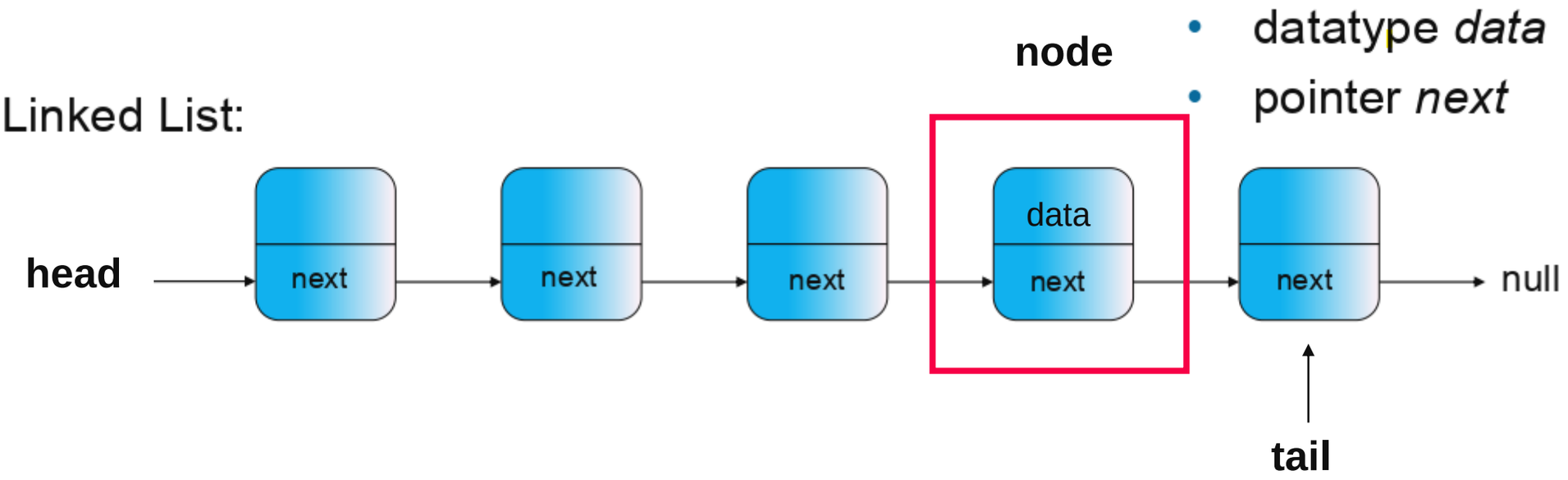
- create(): creates an empty list
- size(): returns the number of elements of the list
- isEmpty(): returns bool, depending whether the list is empty or not
- insert(x, i): inserts the element **x** at the position **i** to the list
- delete(i): deletes element at **i**th position from the list

Single Linked List:



Lists: Implementation

Single Linked List:



- datatype *data*
- pointer *next*

Variables:

- pointer *head*
- pointer *tail*
- int *size*

Operations:

- create():
  pointer *head*, *tail* =null;
  int *size* =0;
- size(): **return** *size***;**
- isEmpty(): **return** (*head*==null);

Lists: Implementation [insert (x,i)]

insert (x, i): inserts the element **x** at the position **i** to the list

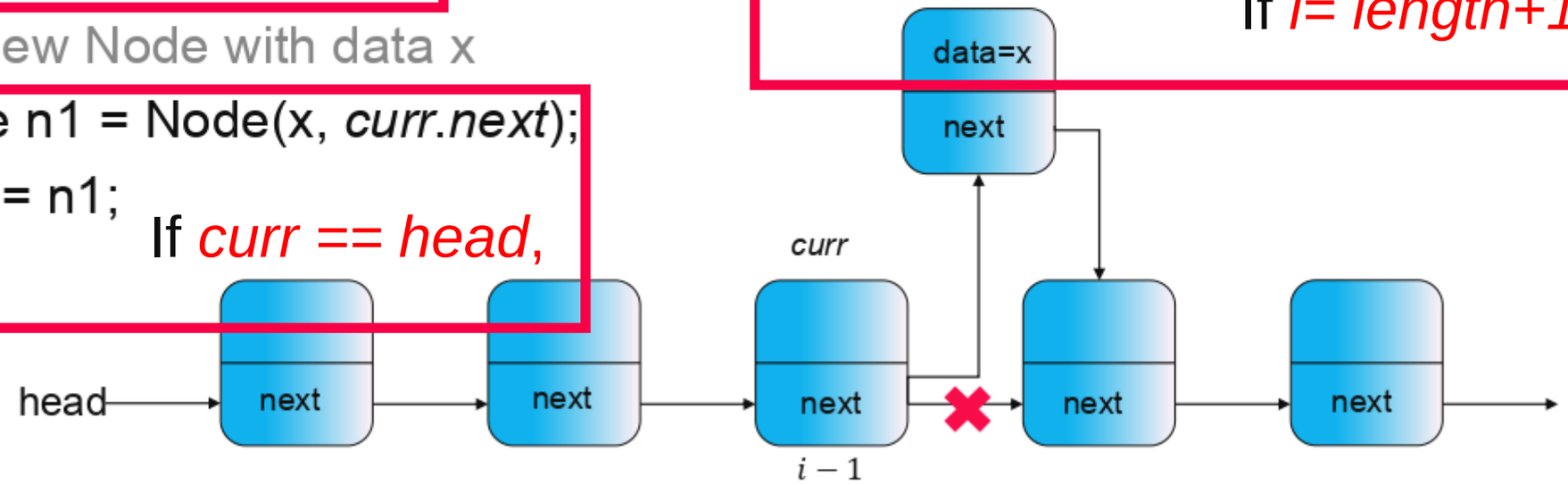
```
//traverse through list to position (i-1)
node curr =  *head;
for(int j=1;j<i;j++)
        curr = *curr.next;
//create new Node with data x
new node n1 = Node(x, curr.next);
curr.next = n1;
size++;
```

If *head* ==null,

//Assumes i ≤ length of list
//changes position of later elements

If *i= length+1*,

If *curr == head*,



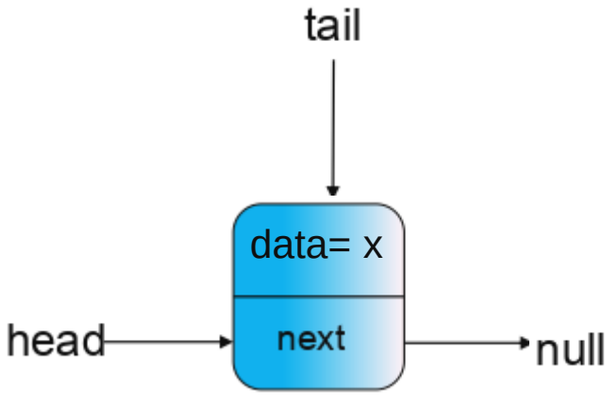
Be careful with edge cases → exercise session

Lists: Implementation [insert (x,i)]

insert (x, i): inserts the element **x** at the position **i** to the list

Edge case)

```
if (head==null) // the list is empty
    new node n1 = Node(x, NULL);
    head = n1;
    tail = n1;
    size ++;
```

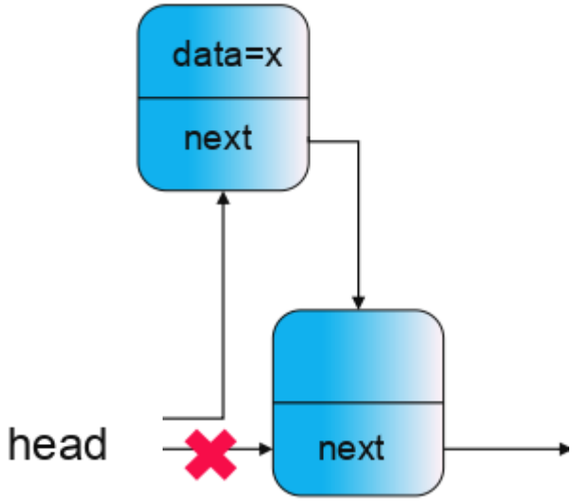


Lists: Implementation [insert (x,i)]

insert (x, i): inserts the element **x** at the position **i** to the list

Edge case)

```
if (i==1) // updating head
    new node n1 = Node(x, head);
    head = n1;
    size ++;
```

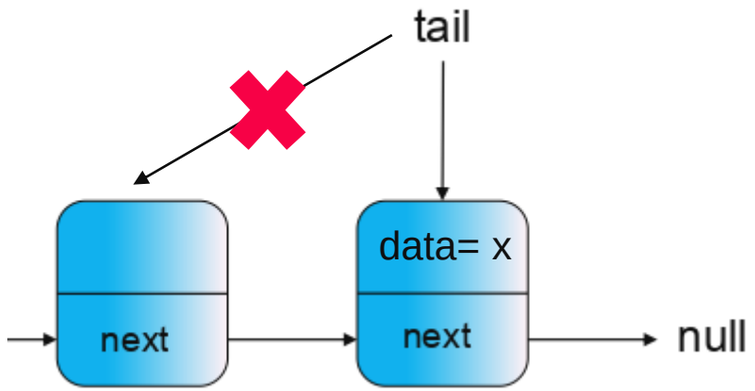


Lists: Implementation [insert (x,i)]

insert (x, i): inserts the element **x** at the position **i** to the list

Edge case)

```
if (i==size+1) // updating tail
    new node n1 = Node(x, NULL);
    prev_tail=*tail;
    pre_tail.next= n1;
    tail=n1;
    size ++;
```



(Or Equivallently, insert the step for the *tail* updating at the end of the current code)

Lists: Implementation [insert (x,i)]

insert (x, i): inserts the element **x** at the position **i** to the list

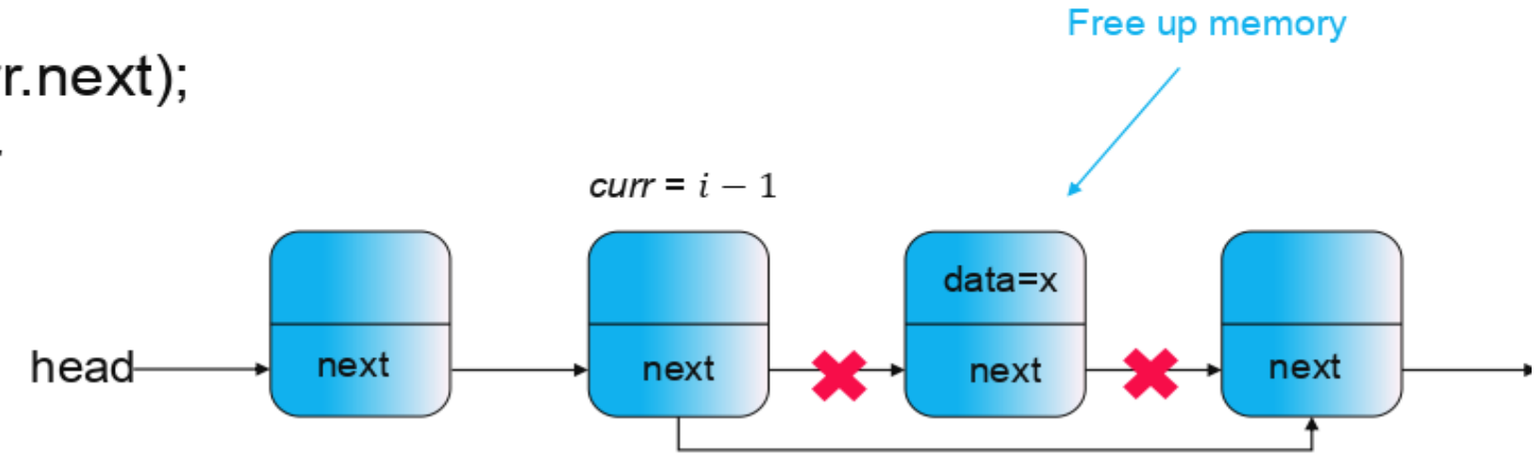
```
if (i<1 or i>size+1) return error;

if (head==null) // the list is empty
        new node n1 = Node(x, NULL);
        head = n1;
        tail = n1;
        size ++;

if (i==1) // updating head
        new node n1 = Node(x, head);
        head = n1;
        size ++;

else
        //traverse through list to position (i-1)
        node curr =  *head;
        for(int j=1;j<i;j++)
                curr = *curr.next;
        //create new Node with data x
        new node n1 = Node(x, curr.next);
        curr.next = n1;
        if (i==size+1) tail=n1; // updating tail
        size++;
```

Lists: Implementation [delete(i)]

delete (i): deletes element at ith position from the list

```
//traverse through list until we find node we want to delete
node curr =  *head;
for(int j=1;j<i;j++)
        curr = *curr.next;
//set new pointers
datatype tmp = data.(*curr.next);
curr.next = curr.next.next;
size--;
return tmp;
```

//Assumes i ≤ length of list
//changes position of later elements

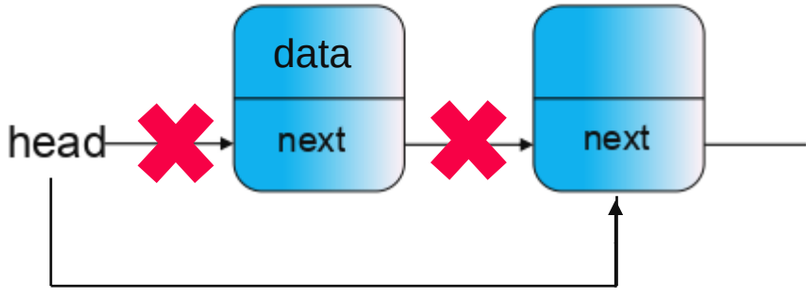


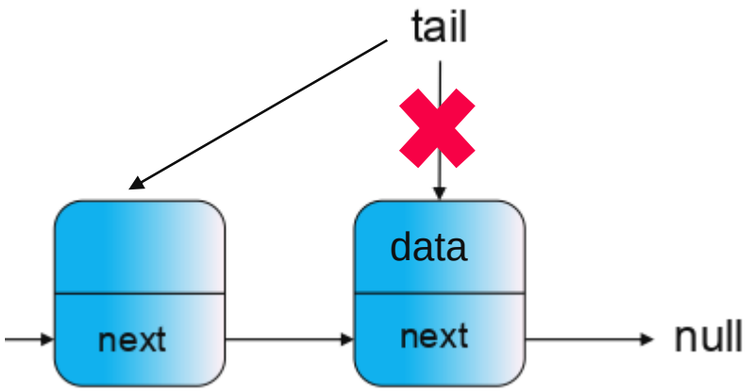
Be careful with edge cases → exercise session

Lists: Implementation [insert (x,i)]

delete (i): deletes element at ith position from the list

```
if (i<1 or i>size) return error;

if (i==1) // updating head
        datatype tmp= data.(*head);
        head = next.(*head);
        size --;
        return tmp;

if (i==size) // updating tail
```





Lists: Implementation [insert (x,i)]

delete (i): deletes element at ith position from the list

```
if (i<1 or i>size) return error;

if (i==1) // updating head
        datatype tmp= data.(*head);
        head = next.(*head);
         if (i==size) tail=NULL ; // updating tail
        size --;
        return tmp;

else
        //traverse through list to position (i-1)
        for(int j=1;j<i;j++)
                curr = *curr.next;
        //set new pointers
        datatype tmp = data.(*curr.next);
        curr.next = curr.next.next;
        if (i==size) tail=curr ; // updating tail
        size--;
        return tmp;
```